

Relatório Técnico: Resiliência e Engenharia de Caos em Arquitetura de Microsserviços

Grupo: Caio Pereira Lapa e Tiago Artém dos Santos

1. Introdução e Arquitetura

Este relatório documenta a execução de testes de Engenharia de Caos em uma arquitetura de microsserviços provisionada em um cluster Kubernetes local (K3d). O objetivo principal é validar as táticas de resiliência e tolerância a falhas do sistema sob condições extremas de estresse.

A arquitetura base é composta por:

- **API Gateway (Python/FastAPI):** Responsável por orquestrar as requisições, aplicando políticas de *Retry* e *Timeout*.
- **Microsserviço de Rotas (C#):** Responsável pelo processamento das regras de negócio.
- **Camada de Persistência (PostgreSQL):** O banco de dados é instanciado nativamente pelo Kubernetes através do manifesto de *Deployment* (*deployments.yaml*), utilizando a imagem oficial *postgres:15-alpine*. Essa estratégia de infraestrutura dispensa a manutenção de um *Dockerfile* dedicado para o banco, delegando a injeção de credenciais e a configuração de portas diretamente às variáveis de ambiente gerenciadas pelo cluster.
- **Orquestração e Caos:** *Kubernetes* para gerenciamento de contêineres e *Chaos Mesh* para injeção declarativa de falhas.

2. Validação de Requisitos Técnicos

Antes da execução dos cenários de caos, a arquitetura foi avaliada e configurada para atender rigorosamente aos critérios técnicos estabelecidos:

2.1. Validação de Regras de Negócio

A restrição logística estipulando que **não deve ser possível voltar para Alegre, nem sair de Castelo**, foi validada com sucesso. A arquitetura demonstrou capacidade de processar essa regra no microsserviço C# e repassar a violação

adequadamente via *Gateway* em *Python*, degradando de forma elegante sem retornar erros genéricos de servidor (HTTP 500).

2.2. Implementação da Tolerância a Falhas

A resiliência do sistema foi implementada através de padrões estabelecidos para microsserviços:

- **Retry:** Configurado via biblioteca *tenacity* para realizar 3 tentativas com 2 segundos de intervalo em caso de indisponibilidade momentânea da API C#.
- **Timeout:** Estabelecido o limite rígido de 2.0 segundos (*httpx.Timeout(2.0)*) para abortar requisições ociosas, protegendo a estabilidade do *Gateway* contra gargalos de rede.
- **Circuit Breaker (Disjuntor de Falhas):** O microsserviço em C# implementa internamente o padrão *Circuit Breaker* através da biblioteca *Polly*. A política foi configurada para interceptar exceções não tratadas e "desarmar" o circuito após 3 falhas consecutivas, mantendo-o aberto por 10 segundos. Isso protege o sistema de um efeito cascata durante falhas severas, retornando o *Status Code* HTTP 503 instantaneamente.
- **Isolamento de Recursos (Limites):** Configuração de *requests* (100m) e *limits* (250m) no *Kubernetes*, garantindo que o alto consumo de CPU nos contêineres não afete o cluster globalmente.

3. Matriz de Experimentos de Caos

Experimento 1: Falha de Instância (PodChaos na API)

- **Estado Estável:** O sistema opera com 2 réplicas do microsserviço C# saudáveis (*Running*), processando requisições logísticas com tempo de resposta em milissegundos e retornando status 200.
- **Hipótese:** A interrupção abrupta de uma instância C# não deve causar indisponibilidade ao usuário final, ocorrendo o redirecionamento para a réplica sobrevivente.
- **Configuração do Ataque:** Manifesto PodChaos (*02-pod-chaos.yaml*) com ação *pod-kill* direcionado via *label* ao *rota-api-csharp*, duração de 30 segundos.
- **Resultado Observado:** O *Kubernetes* identificou a queda (*Terminating*) e iniciou o reprovisionamento instantâneo. A requisição foi interceptada e reenviada internamente, retornando JSON de sucesso.

```
calo_lapa@DESKTOP-L838CCP:~/trabalho-sistemas-distribuidos$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
gateway-api-python-78f479595c-q2stj 1/1     Running   0           50m
gateway-api-python-78f479595c-xshfw 1/1     Running   0           50m
postgres-deployment-866db54cdf-hbmw4 1/1     Running   1 (64m ago) 18h
rota-api-csharp-647cf7664d-grgmw 1/1     Running   0           14m
rota-api-csharp-647cf7664d-k6w5s 1/1     Running   0           97s
rota-api-csharp-647cf7664d-grgmw 1/1     Terminating 0           4m
rota-api-csharp-647cf7664d-grgmw 1/1     Terminating 0           4m
rota-api-csharp-647cf7664d-9c99f 0/1     Pending   0           0s
rota-api-csharp-647cf7664d-9c99f 0/1     Pending   0           0s
rota-api-csharp-647cf7664d-9c99f 0/1     ContainerCreating 0           0s
rota-api-csharp-647cf7664d-9c99f 1/1     Running   0           1s

calo_lapa@DESKTOP-L838CCP:~/trabalho-sistemas-distribuidos$ kubectl apply -f
chaos-experiments/02-pod-chaos.yaml
podchaos.chaos-mesh.org/pod-kill-rota-api created
calo_lapa@DESKTOP-L838CCP:~/trabalho-sistemas-distribuidos$
```

```
localhost:8080/api/gateway/rota?origem=Vitoria&destino=Vila%20Velha

Estilos de formatação ☐

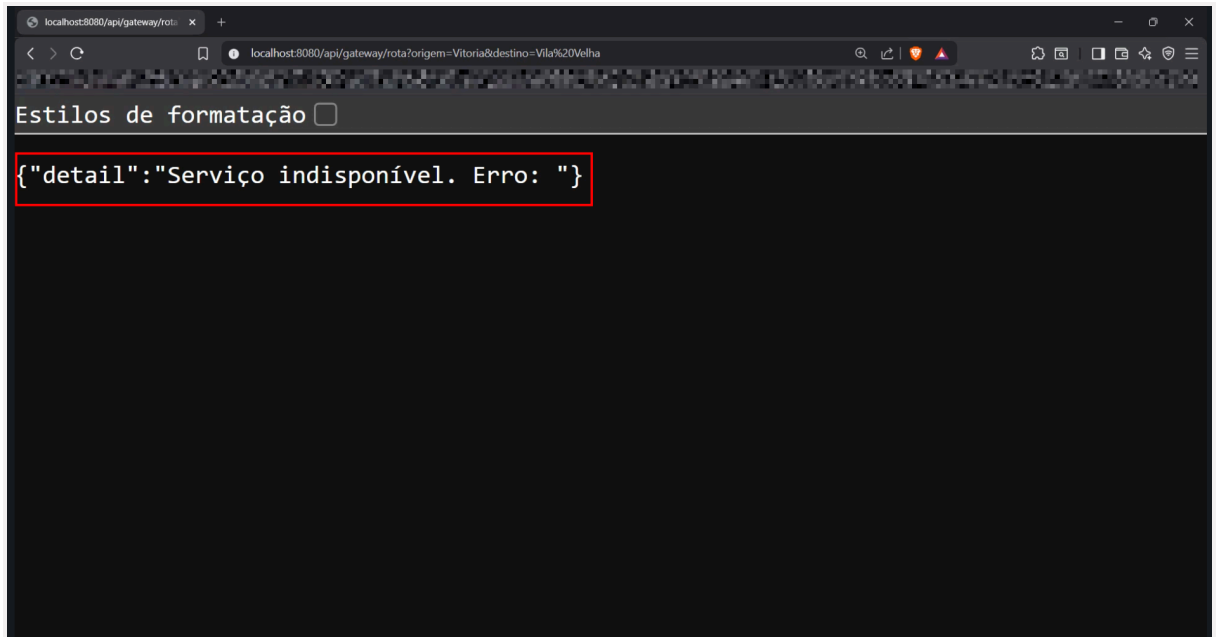
{"origem":"Vitoria","destino":"Vila Velha","distancia_km":292,"status":"Sucesso"}
```

- **Ações Corretivas:** A mitigação ocorre via infraestrutura pelo *ReplicaSet* (replicas: 2) e no código Python através do padrão *Retry* (tenacity).

Experimento 2: Latência Extrema de Rede (NetworkChaos)

- **Estado Estável:** A comunicação entre Gateway e API ocorre sem retenção de pacotes ou enfileiramento pendente.
- **Hipótese:** A injeção de latência superior a 2.0s forçará o Gateway a abortar a requisição proativamente, impedindo o bloqueio de recursos.
- **Configuração do Ataque:** Manifesto NetworkChaos (01-network-chaos.yaml) injetando `latency: 2500ms` nos Pods do C#, duração de 5 minutos.

- **Resultado Observado:** A requisição estourou o limite de tempo configurado. O Gateway interrompeu a conexão e retornou "503 Serviço indisponível".

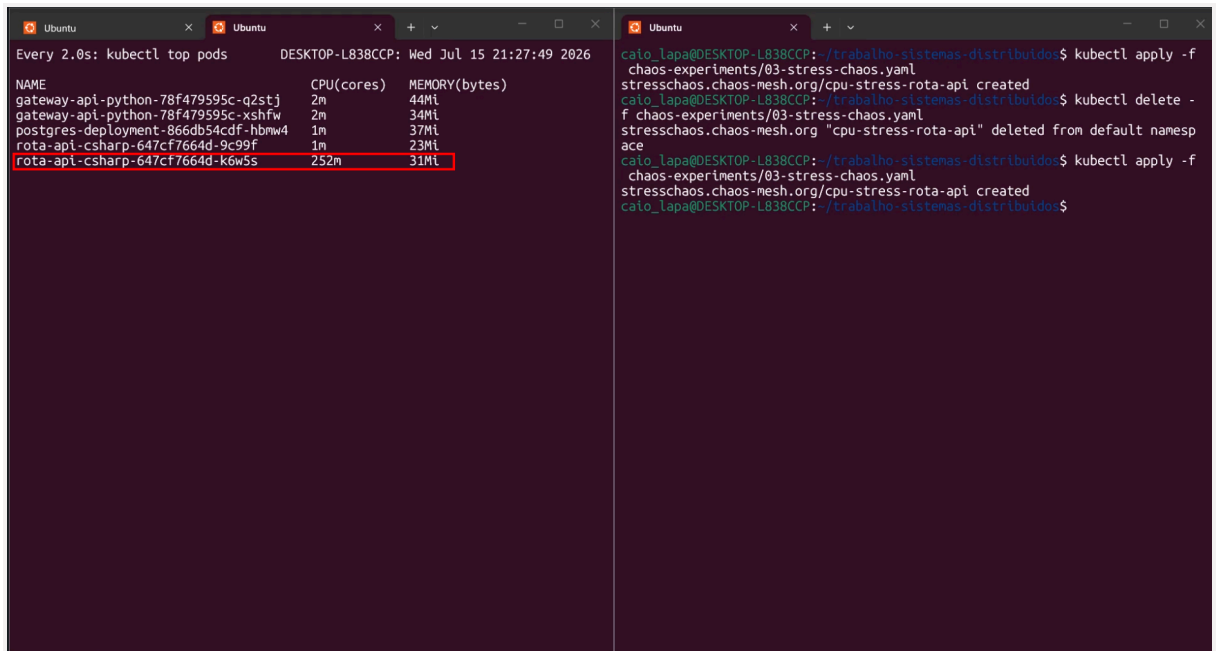


- **Ações Corretivas:** Mitigação executada no Gateway através do *Timeout* rígido de 2.0 segundos, complementada pelo *Circuit Breaker* no C# para evitar saturação de *threads*.

Experimento 3: Exaustão de Recursos - CPU (StressChaos)

- **Estado Estável:** O consumo base de CPU dos Pods da API C# oscila entre 1m e 5m.
- **Hipótese:** Um ataque de estresse levará o contêiner ao limite configurado, mas o *throttling* do Kubernetes impedirá a queda do nó hospedeiro.
- **Configuração do Ataque:** Manifesto **StressChaos** (`03-stress-chaos.yaml`) injetando `load: 100` em 2 *workers* na API C#, duração de 60 segundos.
- **Resultado Observado:** A CPU do Pod atingiu aproximadamente 162m. O K8s isolou o processo, impedindo que ultrapassasse 250m. A aplicação

degradou em performance, mas processou a requisição.



The image shows two terminal windows. The left window displays the output of 'kubectl top pods', showing resource usage for several pods. The 'rota-api-csharp' pod is highlighted with a red box, showing 252m CPU and 31Mi memory usage. The right window shows a series of 'kubectl' commands being executed to apply, delete, and re-apply a chaos experiment named 'cpu-stress-rota-api'.

```
Every 2.0s: kubectl top pods      DESKTOP-L838CCP: Wed Jul 15 21:27:49 2026
```

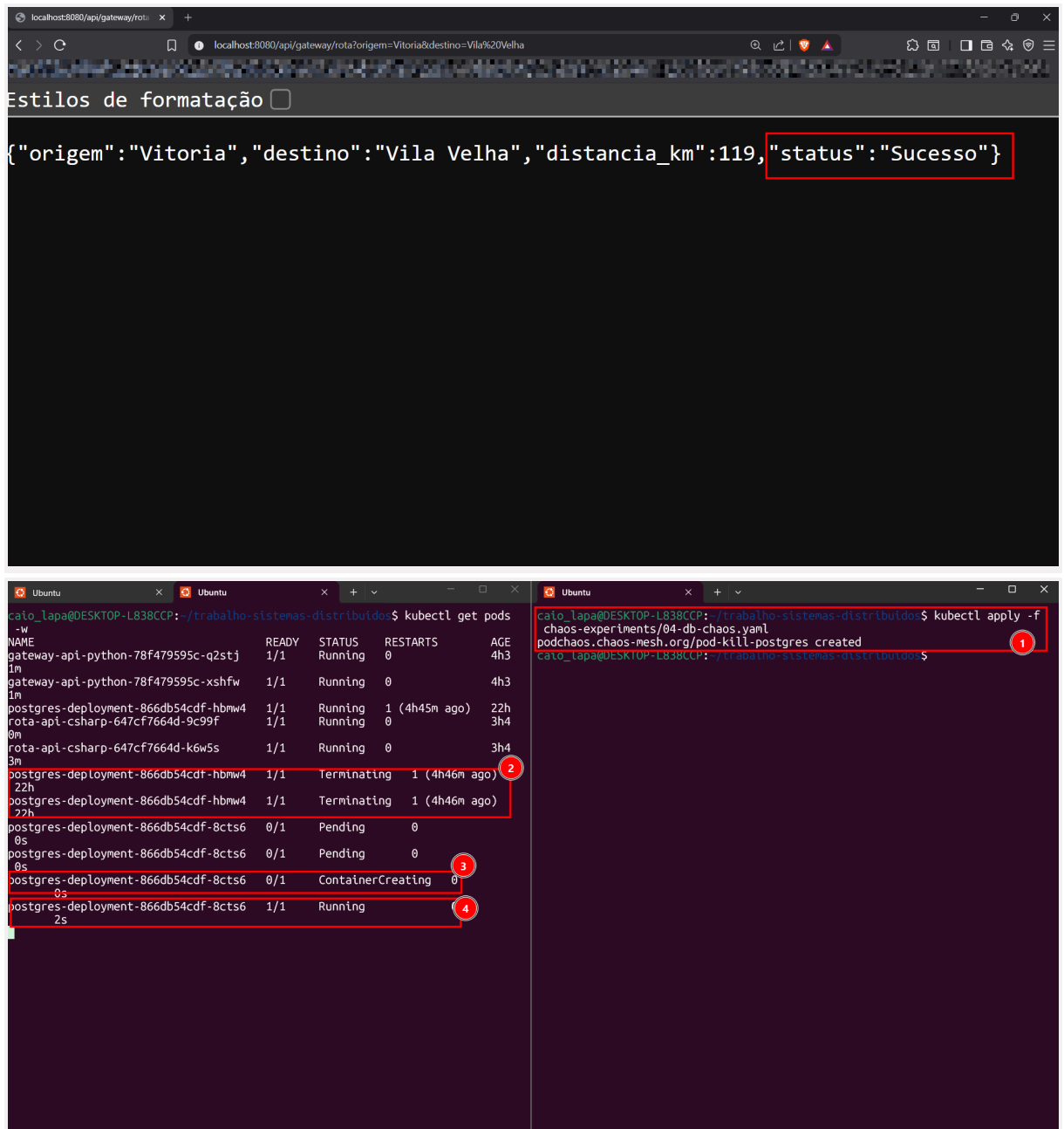
NAME	CPU(cores)	MEMORY(bytes)
gateway-api-python-78f479595c-q2stj	2m	44Mi
gateway-api-python-78f479595c-xshfw	2m	34Mi
postgres-deployment-866db54cdf-hbmw4	1m	37Mi
rota-api-csharp-647cf7664d-9c99f	1m	23Mi
rota-api-csharp-647cf7664d-k6w5s	252m	31Mi

```
calo_lapa@DESKTOP-L838CCP: /trabalho-sistemas-distribuido$ kubectl apply -f chaos-experiments/03-stress-chaos.yaml
stresschaos.chaos-mesh.org/cpu-stress-rota-api created
calo_lapa@DESKTOP-L838CCP: /trabalho-sistemas-distribuido$ kubectl delete -f chaos-experiments/03-stress-chaos.yaml
stresschaos.chaos-mesh.org "cpu-stress-rota-api" deleted from default namespace
calo_lapa@DESKTOP-L838CCP: /trabalho-sistemas-distribuido$ kubectl apply -f chaos-experiments/03-stress-chaos.yaml
stresschaos.chaos-mesh.org/cpu-stress-rota-api created
calo_lapa@DESKTOP-L838CCP: /trabalho-sistemas-distribuido$
```

- **Ações Corretivas:** Proteção implementada via *Resource Quotas* no manifesto de *Deployment* (*requests: 100m, limits: 250m*).

Experimento 4: Falha na Camada de Persistência (PodChaos no Banco)

- **Estado Estável:** O banco de dados PostgreSQL opera sem reinicializações, persistindo dados.
- **Hipótese:** A queda da instância do banco de dados não deve gerar falha em cascata nas rotas lidas em memória/regras de domínio isoladas.
- **Configuração do Ataque:** Manifesto PodChaos (*04-db-chaos.yaml*) com ação *pod-kill* contra o *postgres-deployment*, duração de 60 segundos.
- **Resultado Observado:** Durante a recriação do banco, as requisições de cálculo logístico retornaram sucesso, confirmando o desacoplamento das regras de negócio em relação a consultas síncronas contínuas.



- **Ações Corretivas:** Resiliência garantida por arquitetura baseada em regras de domínio na camada de aplicação, sem dependência rígida do banco para operações de cálculo restritivo.

4. Conclusão

A bateria de testes executada confirmou a eficácia da arquitetura proposta e das estratégias de resiliência implementadas. Por meio da aplicação de princípios de Engenharia de Caos, foi possível validar empiricamente que o sistema é capaz de tolerar a indisponibilidade abrupta de instâncias, evitar o esgotamento de recursos em cascata causado por falhas de rede mediante a utilização de *timeouts* curtos e isolar processos potencialmente prejudiciais por meio da delimitação de recursos.

Dessa forma, a matriz de testes demonstrou que o sistema de roteamento manteve seu funcionamento conforme o esperado diante de cenários de instabilidade de rede, falhas em microsserviços e indisponibilidade dos serviços de persistência. A atuação conjunta dos mecanismos de **Retry, Timeout, Circuit Breaker** e limitação de recursos proporcionou um comportamento consistente e previsível, garantindo a continuidade do serviço mesmo na presença de falhas.

Dessa forma, os resultados obtidos evidenciam que a arquitetura atende aos requisitos de **Tolerância a Falhas** e **Estado Estável** definidos, demonstrando sua capacidade de manter a disponibilidade e a estabilidade operacional em cenários adversos.